

FedWatch AI 用户使用手册

本手册面向希望了解项目结构、复刻预测结果或深入阅读代码的读者。
作者：余青锋 | 仓库：<https://github.com/MessiahYu/fed-agent-project>
(<https://github.com/MessiahYu/fed-agent-project>)

一、项目是什么（60 秒速览）

本项目预测**美联储 FOMC 议息会议的利率决策方向**（加息 / 降息 / 维持不变）。

核心做法是将四路信号融合： 1. **LLM 语义分**：用 DeepSeek 分析每次 FOMC 声明的鹰鸽程度（-1.0 极鸽 → +1.0 极鹰） 2. **VAR 计量信号**：用向量自回归模型，基于利率/通胀/失业率历史数据预测方向 3. **市场信号**：收益率曲线斜率（10Y-2Y） 4. **政治压力信号**：EPU 指数 + VIX 的滚动百分位

系统从 2000 年以来的 217 次 FOMC 会议中学习，在 2025-2026 年的 12 次最新会议上验证。最终版本（V5）在这 12 次中预测正确 11 次（**91.7%**）。

二、五个版本一句话总结

版本	一句话	最终测试集准确率
V1	CrewAI 5智能体实时单次预测（含主席蒸馏、影响网络分析）	无批量统计
V2	引入历史批量回测 + 网格搜索权重优化	75% (9/12)
V3	修复时间固定效应（分层随机 80/20 划分）	75% (9/12)
V4	修复同期信息泄露（LLM 信号滞后一期）	91.7% (11/12)
V5	V3 + V4 同时修复（方法论最完整版）	91.7% (11/12)

三、目录结构

```

fed-agent-project/
├── .env                                # 🚨 API 密钥（不提交 git）
├── config.yml                          # 所有配置（模型、阈值、最优权重等）
├── requirements.txt                    # Python 依赖列表（pip install -r 一行安装）
├── fed_watch.db                        # SQLite 数据库（含历史分析结果，已预填充）
├── prototypes/                        # 最早期的验证脚本
│   ├── test.py                        # 环境连通性测试
│   ├── check_brain.py                 # DeepSeek API 连通验证
│   ├── agent_v1.py                    # 最早的单智能体原型
│   └── agent_v2.py                    # 加了 Pydantic 结构化输出
├── v1/                                # V1: 5 智能体实时预测流水线
│   ├── agent1_data_perception.py      # 智能体1: 抓取美联储官网声明 + FRED 数据
│   ├── agent2_semantic.py             # 智能体2: LLM 语义分析（鹰鸽打分）
│   ├── agent3_decision.py             # 智能体3: 决策推演（VAR + 融合）
│   ├── agent4_chair_distill.py        # 智能体4: 历任主席行为蒸馏
│   ├── agent5_influencers.py         # 智能体5: 外部影响网络分析
│   └── pipeline.py                    # 主入口: 串联 5 个智能体
├── v2/                                # V2: 历史批量回测框架
│   ├── historical_fomc.py              # 数据收集: 抓取历史声明并写库
│   ├── agent2_v2_semantic_history.py  # 批量语义分析（217 次声明）
│   ├── agent5_v2_political_quant.py   # 政治压力信号批量计算
│   ├── weight_optimizer.py            # 核心: 网格搜索最优权重
│   ├── backtest.py                   # VAR 走一步回测 + FRED 数据拉取
│   └── final_test_eval.py             # V2 最终测试集评估（12 次）
├── v3/                                # V3: 分层随机 80/20 划分
│   ├── split_strategy.py              # 重新划分数数据库 split 列
│   ├── final_test_eval_v3.py          # V3 最终测试集评估
│   └── metrics_summary.py             # P/R/F1 全面指标汇总
├── v4/                                # V4: LLM 信号滞后一期
│   ├── weight_optimizer_v4.py         # 含 load_feature_matrix_v4() (shift(1))
│   └── final_test_eval_v4.py          # V4 最终测试集评估
├── v5/                                # V5: V3 + V4 双重修复
│   ├── weight_optimizer_v5.py         # 含 load_feature_matrix_v5() (先 lag 后随机划分)
│   └── final_test_eval_v5.py          # V5 最终测试集评估
├── utils/
│   ├── db.py                          # SQLite 连接与表初始化
│   └── config.py                       # config.yml 读取工具
├── dashboard/
│   └── app.py                          # Streamlit 可视化仪表盘（废弃）
├── docs/
│   ├── 用户使用手册.md                # 本文件
│   ├── 项目总说明文档.md             # 项目总览（含五代迭代树状图）
│   └── v1_system_doc.md                # V1 系统文档

```

└─ v2_system_doc.md	# V2 系统文档
└─ v3_system_doc.md	# V3 系统文档
└─ v4_system_doc.md	# V4 系统文档
└─ v5_system_doc.md	# V5 系统文档

四、数据库结构（三张表）

fed_watch.db 已预填充，克隆仓库后无需重建。

表名	记录数	关键字段	用途
fomc_history	218 条	meeting_date , chair , actual_decision , split , cpi_yoy , unemployment , fed_funds_rate	历史会议基本信息 + 决策 标签
semantic_history	217 条	meeting_date , hawkish_score , inflation_concern , growth_concern	LLM 对每次声明的情绪打 分
political_quant_monthly	320 条（月频）	month , epu , vix , dgs2 , dgs10 , political_score , market_signal	政治压力 + 市场信号（月 频）

五、环境配置

5.1 前置要求

要求	说明
Anaconda / Miniconda	推荐 Anaconda
Python 3.11	项目在 3.11.15 上开发
FRED API Key	免费申请： https://fred.stlouisfed.org/docs/api/api_key.html (https://fred.stlouisfed.org/docs/api/api_key.html)
DeepSeek API Key	仅运行 V1 和重建语义数据库时需要（已预填充可跳 过）

5.2 创建 conda 环境

```
# 创建环境
conda create -n fed-agent python=3.11 -y
conda activate fed-agent

# 一行安装所有依赖
pip install -r requirements.txt
```

5.3 配置 API 密钥

在项目根目录创建 `.env` 文件（不要提交到 git）：

```
DEEPSEEK_API_KEY=sk-xxxxxxxxxxxxxxxxxx
FRED_API_KEY=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

最低配置：只运行最终测试集评估（V2-V5），`FRED_API_KEY` 可选——没有时系统自动降级为使用数据库内预存的历史宏观数据，结果略有差异但仍可运行。
`DEEPSEEK_API_KEY` 仅在运行 V1 或从零重建语义数据库时需要。

六、快速体验：5 分钟看到 V5 预测结果

前提：数据库 `fed_watch.db` 已在仓库中，无需额外准备。有 `FRED_API_KEY` 更好，没有也能运行。

```
git clone https://github.com/MessiahYu/fed-agent-project
cd fed-agent-project
conda activate fed-agent
pip install -r requirements.txt
```

```
# 可选：创建 .env 填入 FRED_API_KEY（没有则自动使用存档数据）
# echo FRED_API_KEY=你的key > .env
```

```
# 直接运行 v5 最终测试集评估（约 1-2 分钟）
python v5/final_test_eval_v5.py
```

预期输出（完整 12 次）：

V5 权重: w_llm=0.40 w_var=0.35 w_market=0.10 w_pol=0.15
 阈值: hike>+0.35 cut<-0.40

基线 (永远 HOLD): 75.0%

V4 准确率:	91.7%	(11/12)
V5 准确率:	91.7%	(11/12)
V5 超越 V4:	+0.0%	

V5 与 V4 预测完全一致，权重差异不影响最终测试集结论。

如果只想查看已保存的结果，直接打开 `v5/v5 final test predictions.csv`。

V1: 实时单次预测 (需要 DeepSeek + FRED)

预期耗时: 30-90 秒 (5 个智能体依次调用 DeepSeek + FRED)

输出内容：- 鹰鸽评分 + 关键证据条目 - VAR 计量信号 - 主席行为蒸馏分析（沃什 vs 历届主席对比） - 外部影响分析（政治压力、国际央行联动） - 最终预测：下次 FOMC 会议决策方向 + 置信度

V2：历史回测 + 最终测试集（需要 FRED，数据库已预填充可跳过前两步）

```
# 【可跳过，已预填充】Step 1: 抓取历史声明，初始化数据库
python v2/historical_fomc.py

# 【可跳过，已预填充，耗时较长】Step 2: 批量语义分析（217 次声明）
python v2/agent2_v2_semantic_history.py

# 【可跳过，已预填充】Step 3: 计算政治压力信号
python v2/agent5_v2_political_quant.py

# Step 4: 运行网格搜索，确定最优权重（约 3-5 分钟）
python v2/weight_optimizer.py

# Step 5: V2 最终测试集评估（12 次）
python v2/final_test_eval.py
```

V3：分层随机划分（需要 FRED）

```
# Step 1: 将数据库中的 split 列改为分层随机 80/20
python v3/split_strategy.py
# 加 --dry-run 只预览，不写库；加 --seed 42 指定随机种子

# Step 2: 在新划分上重跑网格搜索（约 3-5 分钟）
python v2/weight_optimizer.py
# ⚠️ 此时数据库 split 列已是 V3 划分，weight_optimizer.py 读取 split 列

# Step 3: V3 最终测试集评估（12 次）
python v3/final_test_eval_v3.py

# 可选: P/R/F1 全面指标汇总
python v3/metrics_summary.py
```

V4：LLM 信号滞后一期（需要 FRED）

```
# Step 1: V4 网格搜索（含 shift(1)，约 3-5 分钟）
python v4/weight_optimizer_v4.py
# 最优权重自动写入 config.yml → v4_optimal_weights

# Step 2: V4 最终测试集评估（12 次）
python v4/final_test_eval_v4.py
```

V5：双重修复（需要 FRED）

```
# Step 1: V5 网格搜索 (先 lag 后随机划分, 约 3-5 分钟)
python v5/weight_optimizer_v5.py
# 最优权重自动写入 config.yml → v5_optimal_weights

# Step 2: V5 最终测试集评估 (12 次)
python v5/final_test_eval_v5.py
```

八、从零完整复刻（不用预填充数据库）

如果你想验证整个流程，包括数据收集和 LLM 语义分析，按以下顺序操作。
需要 DEEPSEEK_API_KEY + FRED_API_KEY，总耗时约 2-4 小时（LLM 批量分析 217 次声明）。

```
# 1. 初始化数据库并抓取历史会议记录
python v2/historical_fomc.py

# 2. 批量语义分析 (217 次声明, 调用 DeepSeek, 耗时约 1-2 小时)
python v2/agent2_v2_semantic_history.py

# 3. 计算政治压力 + 市场信号 (月频, 约 5 分钟)
python v2/agent5_v2_political_quant.py

# 4. 运行 V2 网格搜索
python v2/weight_optimizer.py

# 5. 运行 V3 划分策略 + V3 网格搜索
python v3/split_strategy.py
python v2/weight_optimizer.py # 此时读取 V3 划分

# 6. V4 网格搜索 (shift(1), 自动重建 V2 时间切割)
python v4/weight_optimizer_v4.py

# 7. V5 网格搜索 (先 lag 后随机划分)
python v5/weight_optimizer_v5.py

# 8. 各版本最终测试集评估
python v2/final_test_eval.py
python v3/final_test_eval_v3.py
python v4/final_test_eval_v4.py
python v5/final_test_eval_v5.py
```

九、config.yml 关键配置说明

config.yml 是唯一配置文件，所有可变参数都在这里，代码中不硬编码业务数值。

```
# 大模型配置
llm:
  model: deepseek/deepseek-chat
  base_url: https://api.deepseek.com

# 数据库
database:
  filename: fed_watch.db

# 各版本最优权重（网格搜索自动写入）
v2_optimal_weights:
  w_llm: 0.45  w_var: 0.20  w_market: 0.15  w_pol: 0.20
  hike_thr: 0.25  cut_thr: -0.35
  val_acc: 0.6562

v3_optimal_weights:
  w_llm: 0.35  w_var: 0.35  w_market: 0.10  w_pol: 0.20
  hike_thr: 0.20  cut_thr: -0.35
  val_acc: 0.6585  split_method: stratified_random_8020

v4_optimal_weights:
  w_llm: 0.45  w_var: 0.30  w_market: 0.15  w_pol: 0.10
  hike_thr: 0.35  cut_thr: -0.35
  val_acc: 0.875  llm_lag: 1

v5_optimal_weights:
  w_llm: 0.40  w_var: 0.35  w_market: 0.10  w_pol: 0.15
  hike_thr: 0.35  cut_thr: -0.40
  val_acc: 0.6667  split_method: stratified_random_8020_llm_lag1
```

十、预期结果速查

运行完成后，各版本的关键输出文件：

文件	内容
v3/v3_final_test_predictions.csv	V2/V3 在 12 次最终测试集的逐次预测
v4/v4_final_test_predictions.csv	V4 在 12 次最终测试集的逐次预测（含 V2 权重对比）
v5/v5_final_test_predictions.csv	V5 在 12 次最终测试集的逐次预测（含 V4 对比）
v3/v3_split_assignments.csv	V3 分层随机划分的完整审计日志（206 次会议分配情况）
config.yml	各版本最优权重（网格搜索自动更新）

核心结论一览（可直接对照 CSV 验证）：

会议	实际	V2	V3	V4	V5
2025-01-29	HOLD	✓	✓	✓	✓
2025-03-19	HOLD	✓	✓	✓	✓

会议	实际	V2	V3	V4	V5
2025-05-07	HOLD	✓	✓	✓	✓
2025-06-18	HOLD	✓	✓	✓	✓
2025-07-30	HOLD	✓	✓	✓	✓
2025-09-17	HOLD	✓	✓	✓	✓
2025-10-29	CUT	X	X	X	X
2025-12-10	CUT	✓	✓	✓	✓
2026-01-28	CUT	X	✓	✓	✓
2026-03-18	HOLD	✓	✓	✓	✓
2026-04-29	HOLD	✓	✓	✓	✓
2026-06-17（沃什）	HOLD	X(HIKE)	X(HIKE)	✓	✓
总准确率		75%	75%	91.7%	91.7%

十一、常见问题

Q：运行时报 ModuleNotFoundError ？

A：确认已激活 fed-agent 环境（conda activate fed-agent），并在项目根目录运行脚本。

Q：FRED 数据拉取超时？

A：FRED API 偶尔限速，等 30 秒后重试即可。也可在 config.yml 的 backtest.var_window 中缩小滚动窗口。

Q：V3 划分和 V4/V5 划分冲突？

A：V4/V5 在代码内显式重建自己的划分（不依赖数据库 split 列），与 V3 不冲突。运行顺序不影响结果。

Q：想修改参数搜索空间？

A：修改 v2/weight_optimizer.py （或 v4/、v5/ 各自对应文件）顶部的 W_GRID、HIKE_GRID、CUT_GRID 列表。

Q：如何查看 V1 的主席蒸馏分析结果？

A：直接运行 v1/pipeline.py，或单独运行 v1/agent4_chair_distill.py（需要 DeepSeek API）。注意主席蒸馏在 V2-V5 中未作为量化特征使用，仅存在于 V1 实时分析流水线。

文档版本：v5-dev 分支 | 余青锋 | 美国东部时间 2026-06-21